

UNIVERSIDAD TECNOLÓGICA NACIONAL

[TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA]

Programación 1

PROYECTO INTEGRADOR

Sistema de gestión de países en Python

Documentación académica y técnica

Estudiantes

Jonatan Angel Valdiviezo, Rodrigo Revollo Torres

Fecha de entrega: 22/06/2026

Índice

Introducción y alcance	
Marco teórico	pág 3
Decisiones técnicas y arquitectura	
Diagrama de flujo de alto nivel	pág 4
Ejemplos de ejecución	pág 5
Dificultades y soluciones	
Conclusiones	
Bibliografía y webgrafía	pág 6
Enlace de entrega	pág 7

1. Introducción y alcance

El proyecto consiste en una aplicación de consola para administrar información básica de países. Resuelve la necesidad de registrar, consultar y analizar datos sin depender de una base de datos ni de bibliotecas externas.

El usuario puede agregar países, actualizar población y superficie, buscar por nombre, filtrar por continente o rangos, ordenar resultados y obtener indicadores estadísticos. Los cambios se conservan en un archivo CSV.

2. Marco teórico

2.1 Programación modular

La programación modular divide un problema en partes con responsabilidades concretas. En este proyecto, la interfaz, las validaciones, las operaciones, las estadísticas y la persistencia se encuentran separadas.

2.2 Listas y diccionarios

La colección completa se representa con una lista. Cada país es un diccionario con nombre, continente, población y superficie. Estas estructuras permiten recorrer, filtrar y ordenar los datos sin definir clases.

2.3 Persistencia en CSV

Persistir significa conservar los datos más allá de la ejecución actual. CSV representa una tabla en texto plano: la primera fila contiene encabezados y cada fila posterior representa un país.

2.4 Validación de datos

La validación impide que información vacía, duplicada o numéricamente inválida llegue al almacenamiento. Se comprueban textos obligatorios, números positivos, rangos coherentes y unicidad del nombre.

2.5 Repository Pattern funcional

Repository establece una frontera para acceder a los datos. El repositorio conoce el CSV; el resto del sistema trabaja con listas y diccionarios. Se aplicó mediante funciones.

3. Decisiones técnicas y arquitectura

3.1 Organización del proyecto

Componente	Responsabilidad
main.py	Punto de entrada
countries_menu.py	Interacción con el usuario
countries_service.py	Operaciones y reglas del sistema
countries_validations.py	Validación y normalización
countries_stats.py	Cálculos estadísticos
countries_repository.py	Lectura y escritura del CSV
data/countries.csv	Datos persistentes

3.2 Decisiones principales

- CSV por simplicidad, portabilidad y adecuación al alcance.
- Ruta calculada desde el proyecto para no depender de la terminal.
- UTF-8 con BOM para conservar acentos y facilitar el uso con planillas.
- Guardado inmediato después de agregar o actualizar.

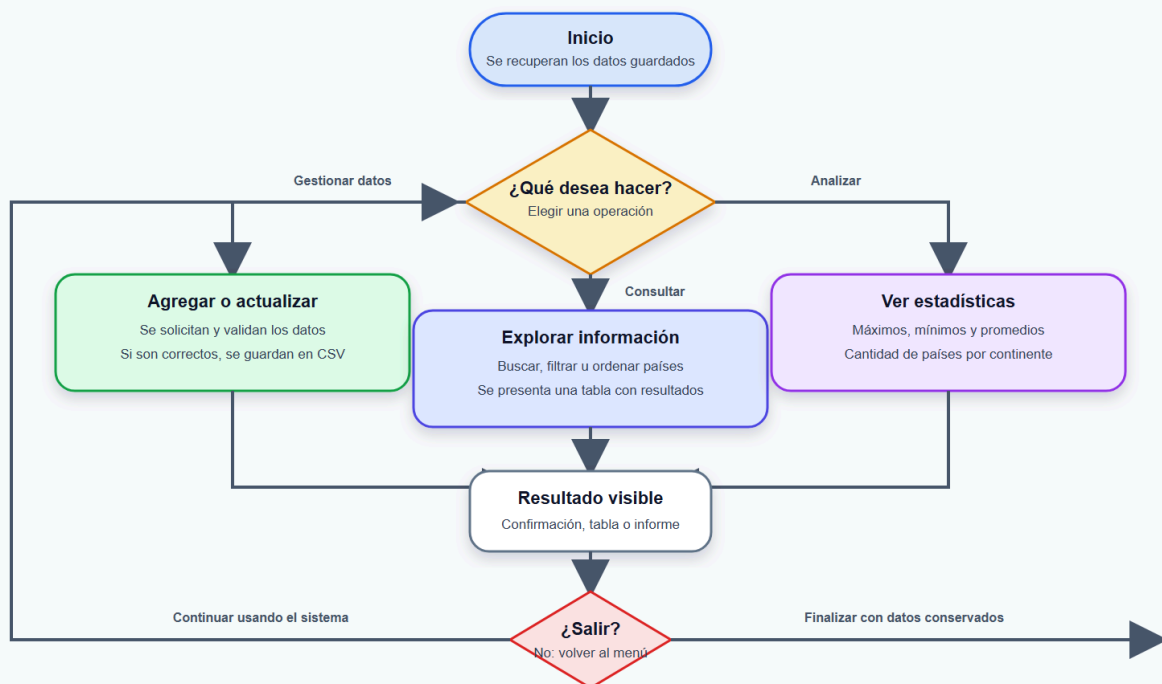
4. Diagrama de flujo de alto nivel

El siguiente flujo explica la experiencia del usuario.

Etapas	Qué sucede
INICIO	Se recuperan los datos guardados
ELECCIÓN	El usuario selecciona una operación
GESTIONAR	Agregar o actualizar → validar → guardar
CONSULTAR	Buscar, filtrar u ordenar → mostrar resultados
ANALIZAR	Calcular estadísticas → presentar informe
RESULTADO	Se muestra una confirmación, tabla o resumen
DECISIÓN	Continuar usando el sistema o finalizar
FIN	Los datos quedan conservados en el CSV

Flujo general — Gestión de países

Vista de alto nivel orientada al usuario



5. Ejemplos de ejecución

5.1 Búsqueda por nombre

```
Seleccione una opción (1-7): 3
```

```
--- BUSCAR PAÍS ---
```

```
Nombre o parte: arg
```

Nombre	Continente	Población	Superficie km²
argentina	americano	23	200.00

```
=====
```

5.2 Estadísticas generales

```
Seleccione una opción (1-7): 6
```

```
--- ESTADÍSTICAS ---
```

```
Mayor población: argentina (23)
```

```
Menor población: argentina (23)
```

```
Población promedio: 23.00
```

```
Superficie promedio: 200.00 km²
```

```
Países por continente:
```

```
- americano: 1
```

6. Dificultades y soluciones

Separar lógica e interfaz

El prototipo mezclaba entrada, salida, validación y modificación. Se resolvió dejando input y print en el menú, mientras los servicios reciben parámetros y devuelven resultados.

Consistencia del CSV

El archivo inicial tenía encabezados y valores inconsistentes. Se estableció un único esquema: name, continent, population y area.

Rutas y codificación

Se calculó la ruta desde el archivo del repositorio y se utilizó UTF-8 con BOM para conservar caracteres especiales.

Validaciones repetidas

Las comprobaciones comunes se centralizaron en un módulo reutilizable con mensajes de error claros.

7. Conclusiones

El desarrollo permitió aplicar funciones, listas, diccionarios, archivos, validaciones, manejo de errores, ordenamiento y estadísticas.

Una aplicación puede mantener una estructura profesional sin incorporar herramientas avanzadas. La claridad de responsabilidades resultó más importante que aumentar la cantidad de abstracciones.

CSV permitió comprender el ciclo completo de los datos: ingreso, validación, transformación, almacenamiento y recuperación.

Las pruebas automatizadas aportaron seguridad durante la reorganización del prototipo y redujeron la posibilidad de introducir errores.

Reflexión final grupal

El resultado es una aplicación simple para el usuario y modular para el equipo al usar listas y diccionarios para manejar los datos sin que el código se vuelva un lío.

Además, ver cómo los datos se guardan de verdad en un archivo CSV y no se borran al cerrar el programa nos dio una idea clara de cómo funciona el software en la realidad. Nos llevamos una base sólida de lógica y la costumbre de escribir código prolijo y bien validado.

8. Bibliografía y webgrafía

- Python Software Foundation. csv — CSV File Reading and Writing.
<https://docs.python.org/3/library/csv.html>
- Python Software Foundation. os — Miscellaneous operating system interfaces.
<https://docs.python.org/3/library/os.html>

- Python Software Foundation. Data Structures.
<https://docs.python.org/3/tutorial/datastructures.html>
- Python Software Foundation. unittest — Unit testing framework.
<https://docs.python.org/3/library/unittest.html>
- Fowler, M. Repository. <https://martinfowler.com/eaCatalog/repository.html>

9. Enlaces de entrega

Repositorio: <https://github.com/Jonatan041101/facultad-tpi/blob/master/main.py>

Video explicativo: <https://drive.google.com/drive/u/0/home>